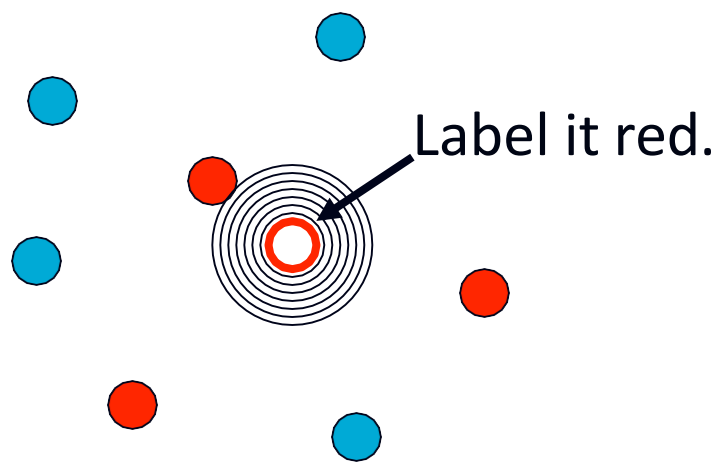
K - Nearest Neighbor

Nearest Neighbor

- One of the simplest of all machine learning classifiers
- label a new point the same as the closest known point



Nearest Neighbor

How Does the K-Nearest Neighbors Algorithm Work?

The K-NN algorithm compares a new data entry to the values in a given data set (with different classes or categories).

Based on its closeness or similarities in a given range (**K**) of neighbors, the algorithm assigns the new data to a class or category in the data set (training data).

Nearest Neighbor

Step #1 - Assign a value to **K**.

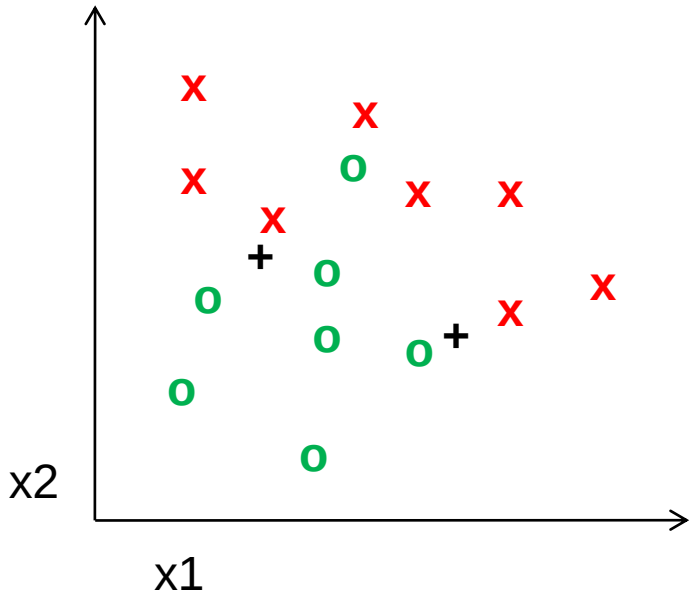
Step #2 - Calculate the distance between the new data entry and all other existing data entries (you'll learn how to do this shortly). Arrange them in ascending order.

Step #3 - Find the **K** nearest neighbors to the new entry based on the calculated distances.

Step #4 - Assign the new data entry to the majority class in the nearest neighbors.

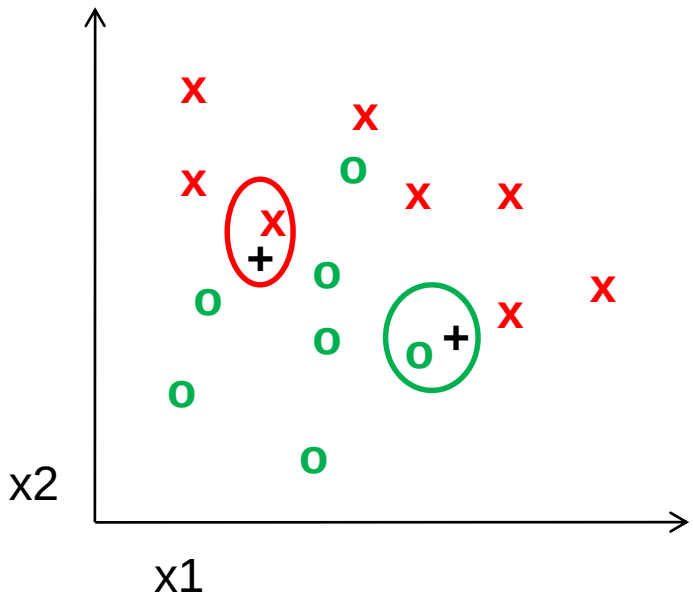
Nearest Neighbor

K-nearest neighbor: predict based on K closest training samples



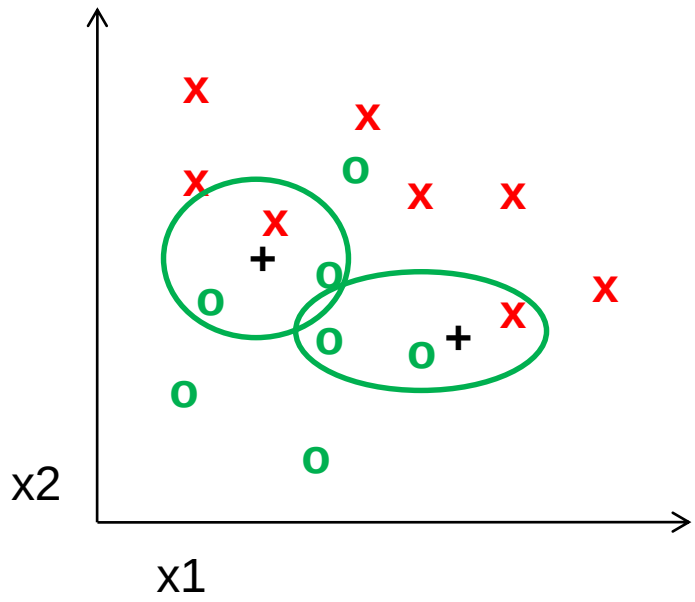
Nearest Neighbor

1-nearest neighbor



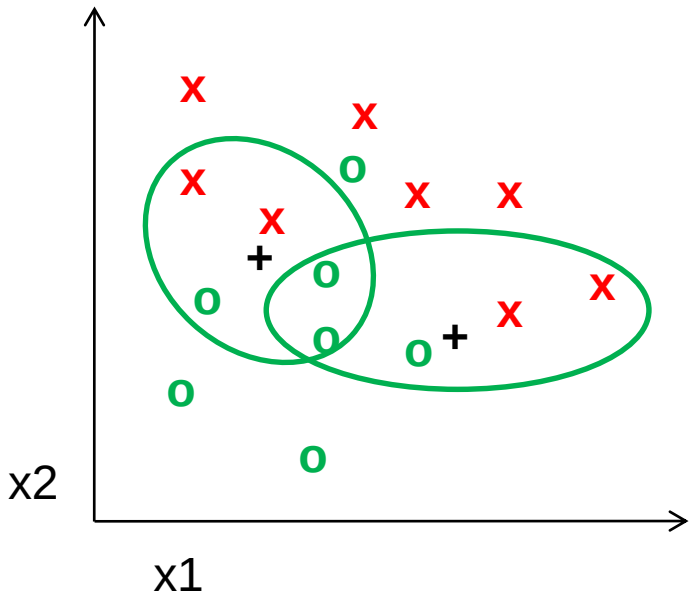
Nearest Neighbor

3-nearest neighbor



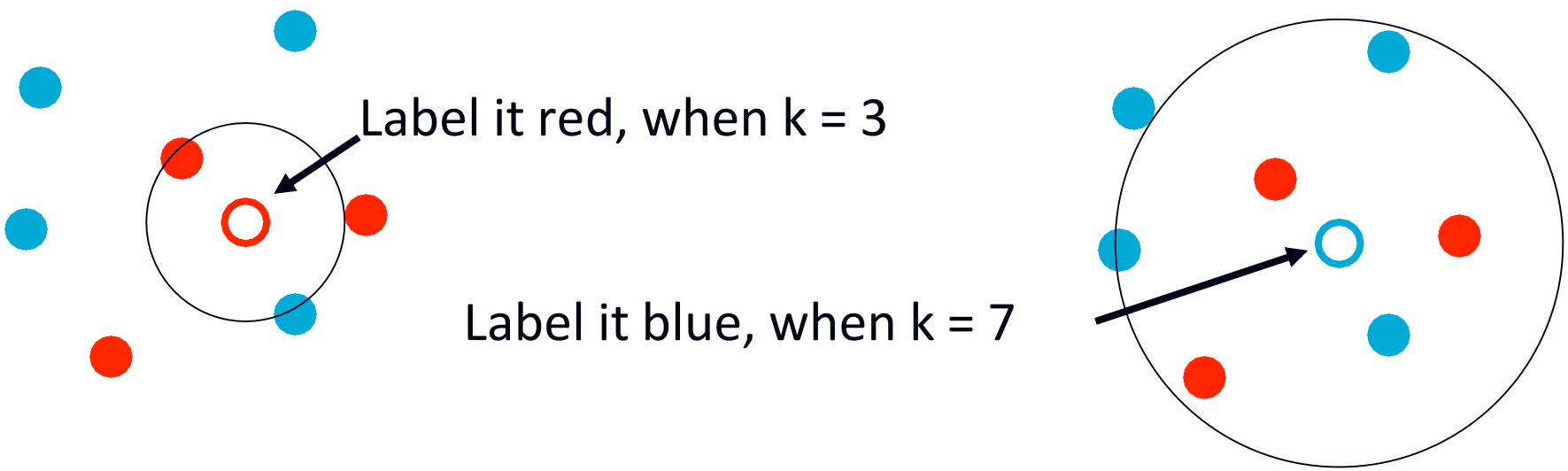
Nearest Neighbor

5-nearest neighbor



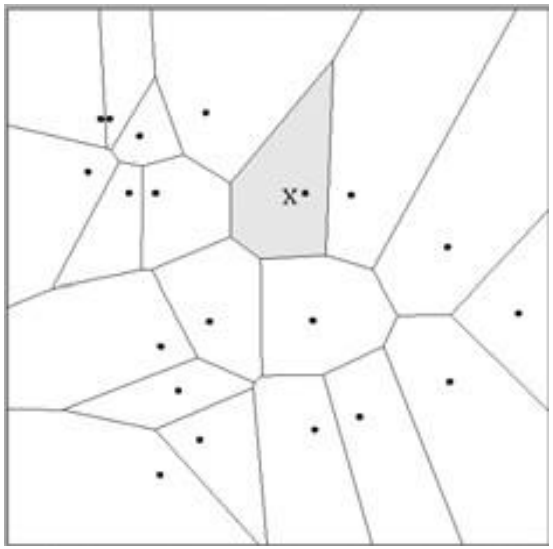
k – Nearest Neighbor

- Generalizes 1–NN to smooth away noise in the labels
- A new point is now assigned the most frequent label of its *k* nearest neighbors

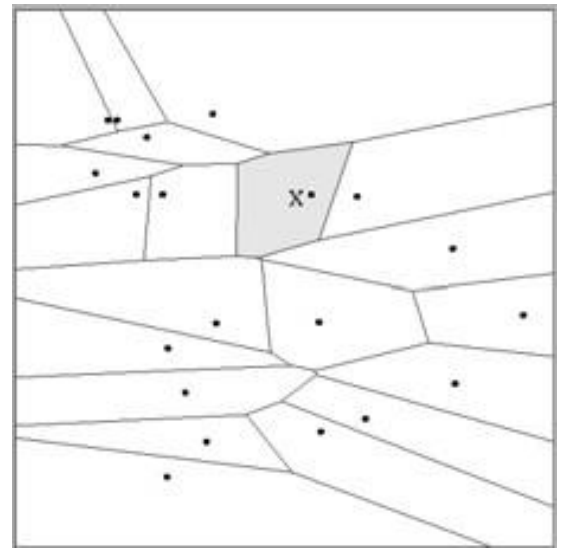


Distance Metrics

- Different metrics can change the decision surface



Dist(**a,b**) $=(a_1 - b_1)^2 + (a_2 - b_2)^2$



Dist(**a,b**) $=(a_1 - b_1)^2 + (3a_2 - 3b_2)^2$

- Standard Euclidean distance metric:
  - Two--dimensional: Dist(a,b) = $\mathrm{sqrt}((a_1 - b_1)^2 + (a_2 - b_2)^2)$
  - Multivariate: Dist(a,b) = $\mathrm{sqrt}(\sum (a_i - b_i)^2)$

Four Aspects of an Instance--Based Learner:

- A distance metric
- How many nearby neighbors to look at?
- A weighting function (optional)
- How to fit with the local points?

Example

BRIGHTNESS	SATURATION	CLASS
40	20	Red
50	50	Blue
60	90	Blue
10	25	Red
70	70	Blue
60	10	Red
25	80	Blue

Euclidean distance formula.

$$\sqrt{(X_2-X_1)^2+(Y_2-Y_1)^2}$$

Where:

- X_2 = New entry's brightness (20).
- X_1= Existing entry's brightness.
- Y_2 = New entry's saturation (35).
- Y_1 = Existing entry's saturation.

Here's the new data entry:

BRIGHTNESS	SATURATION	CLASS
20	35	?

Distance #1

For the first row, d1:

BRIGHTNESS	SATURATION	CLASS
40	20	Red

$$d1 = \sqrt{(20 - 40)^2 + (35 - 20)^2}$$
$$= \sqrt{400 + 225}$$
$$= \sqrt{625}$$
$$= 25$$

Here's what the table will look like after all the distances have been calculated:

BRIGHTNESS	SATURATION	CLASS	DISTANCE
40	20	Red	25
50	50	Blue	33.54
60	90	Blue	68.01
10	25	Red	10
70	70	Blue	61.03
60	10	Red	47.17
25	80	Blue	45

Let's rearrange the distances in ascending order:

BRIGHTNESS	SATURATION	CLASS	DISTANCE
10	25	Red	10
40	20	Red	25
50	50	Blue	33.54
25	80	Blue	45
60	10	Red	47.17
70	70	Blue	61.03
60	90	Blue	68.01

Since we chose 5 as the value of K, we'll only consider the first five rows. That is:

BRIGHTNESS	SATURATION	CLASS	DISTANCE
10	25	Red	10
40	20	Red	25
50	50	Blue	33.54
25	80	Blue	45
60	10	Red	47.17

As you can see above, the majority class within the 5 nearest neighbors to the new entry is **Red**. Therefore, we'll classify the new entry as **Red**.

Here's the updated table:

BRIGHTNESS	SATURATION	CLASS
40	20	Red
50	50	Blue
60	90	Blue
10	25	Red
70	70	Blue
60	10	Red
25	80	Blue
20	35	Red

Advantages of K-NN Algorithm

- It is simple to implement.
- No training is required before classification.

Disadvantages of K-NN Algorithm

- Can be cost-intensive when working with a large data set.
- A lot of memory is required for processing large data sets.
- Choosing the right value of K can be tricky.